

Guia de Apoio: Boilerplate do Pipeline

Estrutura de Código Base para a Atividade de Ciência de Dados

Material Complementar para Implementação Sem Pré-requisito de POO

1 Apresentação

Este documento serve como guia e ponto de partida para o desenvolvimento do seu pipeline. Os blocos de código abaixo representam a estrutura mínima funcional necessária para garantir a reprodutibilidade científica da sua análise exploratória. Seu objetivo é preencher as lacunas marcadas com TODO utilizando as bibliotecas de análise de dados (como `pandas`, `numpy`, `matplotlib` ou `seaborn`).

2 Código Fonte Sugerido

2.1 Ingestão de Dados: `src/extract/extractor.py`

Este script é responsável estritamente por tentar carregar os dados brutos e reportar o diagnóstico inicial da matriz de dados.

```
1      import pandas as pd
2      import os
3
4      def ler_dados_brutos(caminho_arquivo: str) -> pd.DataFrame:
5          """
6          Realiza a leitura automatizada do arquivo CSV.
7          Garante resiliência tratando falhas físicas de IO ou arquivos
8          ausentes.
9          """
10         print(f"[INGESTÃO] Tentando ler o arquivo: {caminho_arquivo}")
11
12         if not os.path.exists(caminho_arquivo):
13             raise FileNotFoundError(f"Erro Crítico: O arquivo especificado
14             não foi encontrado em '{caminho_arquivo}'")
15
16         try:
17             # Dica: Ficar atento ao delimitador (sep) e encoding se necess
18             ário
19             df = pd.read_csv(caminho_arquivo, low_memory=False)
20
21             # LOG DE VOLUMETRIA BRUTA
22             linhas, colunas = df.shape
23             print(f"[SUCCESS] Arquivo carregado com sucesso!")
24             print(f"[LOG] Volumetria Bruta Inicial: {linhas} linhas e {
25             colunas} colunas.\n")
```

```
22         return df
23
24     except Exception as e:
25         print(f"[ERRO] Falha inesperada na leitura do arquivo: {str(e)}")
26     raise e
27
28
```

2.2 Processamento Estatístico: src/transform/cleaner.py

Este script concentra a inteligência estatística e as regras de tratamento de dados (EDA).

```
1      import pandas as pd
2      import numpy as np
3
4      def tratar_dados_ausentes(df: pd.DataFrame) -> pd.DataFrame:
5          """
6          Identifica dados nulos e aplica a estratégia estatística
7          escolhida.
8          """
9          print("[EDA] Iniciando tratamento de dados ausentes...")
10         nulos_por_coluna = df.isnull().sum()
11         print(f"[LOG] Contagem de nulos:\n{nulos_por_coluna[
12         nulos_por_coluna > 0]}\n")
13
14         # TODO: Implementar a estratégia de imputação ou descarte aqui
15         df_tratado = df.copy()
16         return df_tratado
17
18     def filtrar_outliers_iqr(df: pd.DataFrame, coluna_numerica:
19     str) -> pd.DataFrame:
20         """
21         Aplica o método matemático do Intervalo Interquartil (IQR)
22         para isolar anomalias.
23         """
24         print(f"[EDA] Calculando limites de IQR para a coluna: '{
25         coluna_numerica}'")
26
27         # TODO: Implementar o cálculo do IQR e o filtro aqui
28         # Q1 = df[coluna_numerica].quantile(0.25)
29         # Q3 = df[coluna_numerica].quantile(0.75)
30         # IQR = Q3 - Q1
31         # limite_inferior = Q1 - 1.5 * IQR
32         # limite_superior = Q3 + 1.5 * IQR
33
34         df_filtrado = df.copy()
35         return df_filtrado
36
37     def executar_transformacao_completa(df_principal: pd.DataFrame
38     , df_secundario: pd.DataFrame = None) -> pd.DataFrame:
39         """
40         Orquestra a higienização, tipagem, normalização temporal e jun
41         ção lógica.
42         """
43         df = df_principal.copy()
44         df = tratar_dados_ausentes(df)
45
46         # TODO: Chamar o filtro de IQR nas colunas numéricas críticas
47         # df = filtrar_outliers_iqr(df, 'coluna_escolhida')
```

```
42     # TODO: Executar o Merge se o seu tema exigir duas bases
43     return df
44
```

2.3 Visualização Científica: src/visualize.py

Responsável por gerar o gráfico estrito seguindo os princípios de integridade analítica e ética visual.

```
1      import pandas as pd
2      import matplotlib.pyplot as plt
3      import seaborn as sns
4
5      def gerar_visualizacao_cientifica(df_limpo: pd.DataFrame,
6      caminho_saida: str = "grafico_integridade.png"):
7          """
8          Gera e exporta um gráfico analítico respeitando a ética estatística.
9          Eixos iniciados em zero e unidades de medida explícitas.
10         """
11         print("[VISUALIZAÇÃO] Gerando gráfico de integridade visual...")
12
13         plt.figure(figsize=(10, 6))
14
15         # TODO: Escolher a visualização adequada e plotar
16         # sns.scatterplot(data=df_limpo, x='variavel_x', y='variavel_y')
17
18         # Garantindo a integridade visual (Princípio ético de Ciência de Dados)
19         plt.axhline(0, color='gray', linestyle='--', linewidth=0.8)
20         plt.title("Título Científico do Gráfico", fontsize=14,
21         fontweight='bold')
22         plt.xlabel("Variável Independente (Unidade de Medida)",
23         fontsize=11)
24         plt.ylabel("Variável Dependente (Unidade de Medida)", fontsize=11)
25         plt.grid(True, linestyle=':', alpha=0.6)
26
27         plt.tight_layout()
28         plt.savefig(caminho_saida, dpi=300)
29         print(f"[SUCCESS] Gráfico científico exportado em: '{caminho_saida}'\n")
30         plt.close()
```

2.4 Orquestrador Central: src/main.py

O arquivo principal que unifica e executa os scripts de forma linear e automatizada.

```
1      from extract.extractor import ler_dados_brutos
2      from transform.cleaner import executar_transformacao_completa
3      from visualize import gerar_visualizacao_cientifica
4
5      def rodar_pipeline():
```

```

6      print("="*60)
7      print("EXECUTANDO PIPELINE REPRODUTÍVEL DE CIÊNCIA DE DADOS")
8      print("="*60 + "\n")
9
10     CAMINHO_RAW_PRINCIPAL = "data/raw/dataset_principal.csv"
11     CAMINHO_SAIDA_FINAL = "dados_limpos_final.csv"
12
13     try:
14         # 1. Ingestão
15         df_bruto = ler_dados_brutos(CAMINHO_RAW_PRINCIPAL)
16
17         # 2. Transformação Estatística
18         df_limpo = executar_transformacao_completa(df_bruto)
19         df_limpo.to_csv(CAMINHO_SAIDA_FINAL, index=False)
20         print(f"[SUCESSO] Base exportada em '{CAMINHO_SAIDA_FINAL}'\n")
21     )
22
23     # 3. Visualização Científica
24     gerar_visualizacao_cientifica(df_limpo)
25
26     print("Pipeline finalizado com sucesso.")
27
28     except FileNotFoundError as fnf_error:
29         print(f"\n[ERRO DE INFRAESTRUTURA] O pipeline parou: {
30 fnf_error}")
31     except Exception as e:
32         print(f"\n[FALHA GERAL] Erro inesperado: {str(e)}")
33
34     if __name__ == "__main__":
35         rodar_pipeline()

```